

Universidad de San Carlos de Guatemala
Facultad de Ingeniería Escuela de Ciencias y
Sistemas Organización de Lenguajes y
Compiladores 1

Vacaciones de diciembre 2025

Catedrático:

Ing. Mario Bautista

Tutor académico:

Lesther López



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

JavaUSAC

Tabla de Contenido

1. Objetivo General	4
2. Objetivos específicos	4
3. Descripción General	5
Flujo de la aplicación	5
4. Entorno de Trabajo	5
4.1 Editor	5
4.2 Funcionalidades	5
4.3 Características	6
4.4 Herramientas	6
4.5 Reportes	6
4.6 Área de Consola	6
5. Descripción del Lenguaje	7
5.1 Case Insensitive	7
5.2 Comentarios	7
5.3 Tipos de Dato	7
5.4 Secuencias de escape	8
5.5 Operadores Aritméticos	9
5.5.1 Suma	9
5.5.2 Resta	9
5.5.3 Multiplicación	10
5.5.4 División	10
5.5.5 Potencia	11
5.5.6 Módulo	11
5.5.7 Negación Unaria	12
5.6 Operadores Relacionales	12
5.7 Operadores Lógicos	13
5.8 Signos de Agrupación	14
5.9 Precedencia de Operaciones	14
5.10 Caracteres de finalización y encapsulamiento de sentencias	15
5.11 Declaración de variables	15
5.12 Asignación de variables	16
5.13 Casteos	16
5.14 Incremento y Decremento	17

5.15 Sentencias de control	17
5.15.1 Sentencia IF	17
5.15.2 Sentencia Switch	19
5.16 Sentencias Cíclicas	20
5.16.1 While	21
5.16.2 For	22
5.16.3 Do-While	22
5.17 Sentencias de Transferencia	22
5.17.1 Break	22
5.17.2 Continue	23
5.17.3 Return	24
5.18 Vectores	25
5.18.1 Declaración de Vectores	25
5.18.2 Acceso a vectores	25
5.18.3 Asignación de Vectores	25
5.19 Listas Dinámicas	26
5.19.1 Declaración de Listas	26
5.19.2 Acceso a listas	26
5.19.3 Asignación de Listas	27
5.19.4 Append	27
5.19.5 Remove	27
5.21 Funciones	27
5.22 Métodos	28
5.23 Llamadas	29
5.24 Función Print	30
5.25 Función round	30
5.26 Length	31
5.27 ToString	31
5.28 Find	31
5.29 Función START_WITH	32
6. Reportes	32
6.1 Tabla de errores	32

6.2 Tabla de Símbolos	32
6.3 AST	33
6.4 Salidas en consola	33
7. Restricciones	33
8. Entrega	34

1. Objetivo General

Aplicar los conocimientos sobre la fase de análisis léxico, sintáctico y semántico de un compilador para la realización de un intérprete sencillo, con las funcionalidades principales para que sea funcional.

2. Objetivos específicos

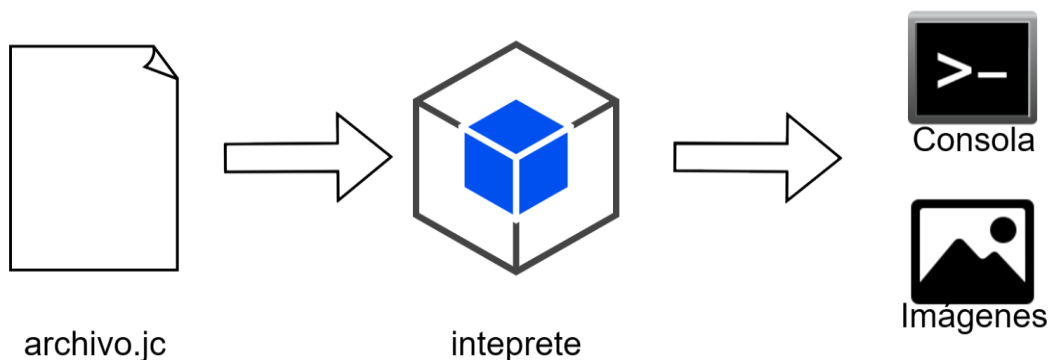
- Reforzar los conocimientos de análisis léxico, sintáctico y semántico para la creación de un lenguaje de programación.
- Aplicar los conceptos de compiladores para implementar un proceso de interpretación de código de alto nivel.
- Construir un árbol de sintaxis abstracta (AST) para la ejecución de código de alto nivel.
- Detectar y reportar errores léxicos, sintácticos y semánticos.

3. Descripción General

El curso de Organización de Lenguajes y Compiladores 1 ha iniciado un proyecto cuyo propósito es desarrollar un nuevo lenguaje de programación que pueda emplearse en los cursos de Introducción a la Programación y Computación 1 y 2. Este lenguaje se basa en los principios fundamentales de la programación y destaca especialmente el trabajo con estructuras de datos.

Por ello, como estudiante de Compiladores 1, se asigna el desarrollo del proyecto JavaUSAC, tomando en cuenta su dominio en análisis léxico, sintáctico y semántico.

Flujo de la aplicación



4. Entorno de Trabajo

4.1 Editor

El editor tendrá como función principal permitir el ingreso del código fuente que será procesado por el sistema. El diseño visual y funcional queda a criterio del estudiante.

4.2 Funcionalidades

- **Nuevo archivo:** El editor debe permitir crear documentos en blanco.
- **Abrir archivos:** Debe ser posible abrir archivos con extensión *.ju*, los cuales se desplegarán en una nueva pestaña mostrando su contenido.
- **Guardar archivo:** El sistema debe permitir guardar los cambios del archivo actualmente abierto.
- **Cerrar pestaña:** Las pestañas pueden cerrarse en cualquier momento; si el archivo tenía cambios sin guardar, estos se perderán.

4.3 Características

- **Manejo de múltiples pestañas:** Se podrán abrir varias pestañas para visualizar, editar o crear distintos archivos. Cada pestaña corresponderá a un archivo independiente.

4.4 Herramientas

- **Ejecutar:** Esta opción debe invocar al intérprete, encargado de realizar los análisis léxico, sintáctico y semántico, así como de ejecutar las instrucciones del programa.

4.5 Reportes

- **Reporte de Errores:** se mostrarán todos los errores encontrados al realizar el análisis léxico, sintáctico y semántico.
- **Generar AST:** se debe generar el ast del último archivo analizado y mostrarlo desde la interfaz.
- **Reporte de Tabla de Símbolos:** Se mostrarán todas las variables, funciones y métodos declarados durante la ejecución del programa.

4.6 Área de Consola

Este espacio mostrará los resultados, mensajes y cualquier salida que produzca el lenguaje durante su ejecución.

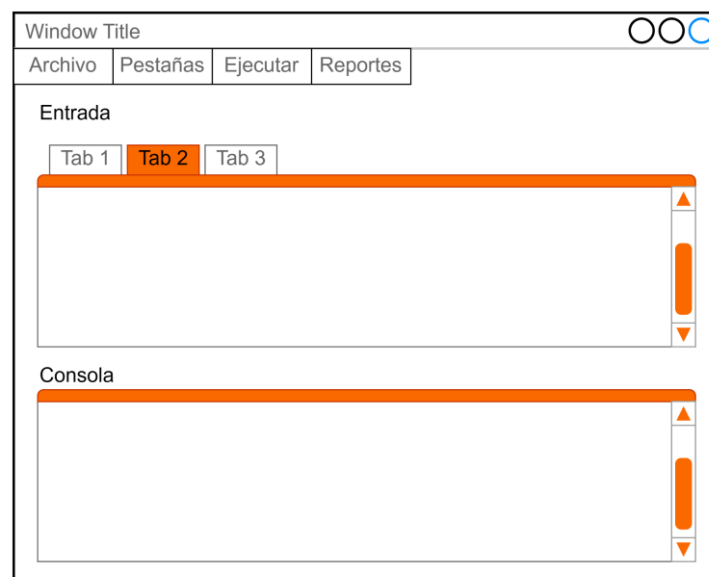


Figura 1. Propuesta de interfaz

5. Descripción del Lenguaje

5.1 Case Insensitive

El lenguaje no distinguirá entre mayúsculas o minúsculas.

```
var a: int = 1;
```

```
Var A :Int = 1;
```

Nota: Ambos casos son lo mismo

//Debe dar error la declaración de "A" ya que la variable "a" ya existe previamente

//int es lo mismo que INT

5.2 Comentarios

5.2.1 Comentarios de una línea

Estos comentarios deberán comenzar con // y terminar con un salto de línea.

5.2.2 Comentarios multilínea

Este comentario comenzará con /* y terminará con */.

```
// Esto es un comentario de una sola línea
```

```
/*
```

```
Esto es un  
comentario multilínea
```

```
*/
```

5.3 Tipos de Dato

Los tipos de dato que soportara el lenguaje en concepto de un tipo de variable se definen a continuación:

Tipo	Definición	Descripción	Ejemplo	Observaciones	Default
Entero	int	Este tipo de dato aceptará solamente números enteros	1, 50, 100, -120, etc	Del -2147483648 al 2147483647	0
Decimal	double	Admite valores numéricos con decimales	1.2, 50.23, -00.34. etc	Se maneja cualquier cantidad de decimales	0.0
Booleano	bool	Admite valores que indican verdadero o falso	true, false	Si se asigna un valor booleano a un entero se tomará como 1 o 0 respectivamente	true

Carácter	char	Tipo de dato que únicamente aceptará un único carácter, y estará delimitado por comillas simples	'a', 'b', 'c', 'E', '1', '&', 'n', etc	En caso de querer escribir comilla simple, se escribirá \ y después la comilla simple \. Si se quiere escribir \ se escribirá \\. Existirá también \n, \t, \r, \".	'\u0000' (carácter 0)
Cadena	String	Es un grupo o conjunto de caracteres que pueden tener cualquier carácter, y este se encontrará delimitado por comillas dobles. “ “	“cadena”, “- cad”	Se permitirá cualquier carácter entre las comillas dobles, incluyendo las secuencias de escape	“” (string vacío)

5.3 Secuencias de escape

Las secuencias de escape se utilizan para definir ciertos caracteres especiales dentro de cadenas de texto. Las secuencias de escape disponibles son las siguientes:

Secuencia	Descripción	Ejemplo
\n	Salto de línea	“Hola\nMundo”
\\	Barra invertida	“C:\miCarpeta”
\”	Comilla doble	“\”esto es una cadena\””
\t	Tabulación	“\tEsto es una tabulación”
\’	Comilla simple	“\’Estas son comillas simples\’”

5.4. Operadores Aritméticos

5.3.1 Suma

Es la operación aritmética que consiste en realizar la suma entre dos o más valores. Para esta se utiliza el signo más (+).

Especificaciones de la operación suma

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

+	Entero	Decimal	Booleano	Carácter	Cadena
Entero	Entero	Decimal		Entero	Cadena
Decimal	Decimal	Decimal		Decimal	Cadena
Booleano					Cadena
Carácter	Entero	Decimal		Cadena	Cadena
Cadena	Cadena	Cadena	Cadena	Cadena	Cadena

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.3.2 Resta

Es la operación aritmética que consiste en realizar la resta entre dos o más valores. Para esta se utiliza el signo menos (-).

Especificaciones de la operación resta

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

-	Entero	Decimal	Carácter
Entero	Entero	Decimal	Entero
Decimal	Decimal	Decimal	Decimal
Carácter	Entero	Decimal	

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.3.3 Multiplicación

Operación aritmética que consiste en sumar un número (multiplicando) tantas veces como indica otro número (multiplicador). El signo para representar la operación es el asterisco (*).

Especificaciones de la operación multiplicación

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

*	Entero	Decimal	Carácter
Entero	Entero	Decimal	Entero
Decimal	Decimal	Decimal	Decimal
Carácter	Entero	Decimal	

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.3.4 División

Operación aritmética que consiste en partir un todo en varias partes, al todo se le conoce como dividendo, al total de partes se le llama divisor y el resultado recibe el nombre de cociente. El operador de la división es la diagonal (/).

Especificaciones de la operación división

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

/	Entero	Decimal	Carácter
Entero	Decimal	Decimal	Decimal
Decimal	Decimal	Decimal	Decimal
Carácter	Decimal	Decimal	

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.3.5 Potencia

Es una operación aritmética de la forma $a^{**}b$ donde a es el valor de la base y b es el valor del exponente que nos indicará cuantas veces queremos multiplicar el mismo número. Por ejemplo $5^{**}3$, $a=5$ y $b=3$ tendríamos que multiplicar 3 veces 5 para obtener el resultado final; $5 \times 5 \times 5$ que da como resultado 125. Para realizar la operación se utilizará el signo ******.

Especificaciones de la operación potencia

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

**	Entero	Decimal
Entero	Entero	Decimal
Decimal	Decimal	Decimal

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.3.6 Módulo

Es una operación aritmética que obtiene el resto de la división de un número entre otro. Para realizar la operación se utilizará el signo **(%)**.

Especificaciones de la operación módulo

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

%	Entero	Doble
Entero	Decimal	Decimal
Decimal	Decimal	Decimal

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.3.7 Negación Unaria

Es una operación que niega el valor de un número, es decir que devuelve el contrario del valor original. Se utiliza el símbolo menos (-).

Especificaciones de la operación negación

A continuación, se especifica en una tabla los resultados que se deberán obtener con esta operación.

-exp	Resultado
Entero	Entero
Decimal	Decimal

Nota: Cualquier otra combinación no especificada en esta tabla es invalida y será considerado un error de tipo semántico.

5.4 Operadores Relacionales

Son los símbolos que tienen como finalidad comparar expresiones, dando como resultado valores booleanos. A continuación, se definen los símbolos que serán aceptados dentro del lenguaje:

Nota: Se pueden realizar operaciones relacionales entre: entero-entero, entero-doble, entero-carácter, doble-entero, doble-carácter, carácter-entero, carácter-doble, carácter-carácter y cualquier otra operación relacional entre entero, doble y carácter

Operador	Descripción	Ejemplo
==	Igualación: compara ambos valores y verifica si son iguales	1==1 25.654 == 54.34
!=	Diferenciación: compara ambos lados y verifica si son distintos	1 != 2 50 != 30
<	Menor que: compara ambos lados y verifica si el derecho es mayor que el izquierdo.	25.5 < 30 50 < 'F'

<=	Menor o igual que: Compara ambos lados y verifica si el derecho es mayor o igual que el izquierdo.	25.5 <= 30 50 <= 'F'
>	Mayor que: compara ambos lados y verifica si el izquierdo es mayor que el derecho.	25.5 > 30 50 > 'F'
>=	Mayor o igual que: Compara ambos lados y verifica si el izquierdo es mayor o igual que el derecho.	25.5 >= 30 50 >= 'F'

5.5. Operadores Lógicos

Son los símbolos que tienen como finalidad comparar expresiones a nivel lógico (verdadero o falso). A continuación, se definen los símbolos que serán aceptados dentro del lenguaje:

Operador	Descripción	Ejemplo
	OR: compara expresiones lógicas y si al menos una es verdadera, entonces devuelve verdadero y en otro caso retorna falso.	true 5<2 Devuelve true
&&	AND: compara expresiones lógicas y si ambas son verdaderas, entonces devuelve verdadero y en otro caso retorna falso.	true && "hola" == "hola" Devuelve true
^	XOR: compara expresiones lógicas y si ambas no son iguales retorna verdadero y en cualquier otro caso retorna false.	true ^ false devuelve true
!	NOT: devuelve el valor inverso de una expresión lógica si esta es verdadera entonces devolverá false, de lo contrario retorna verdadero	!true Devuelve false

5.6. Signos de Agrupación

Los signos de agrupación serán utilizados para agrupar operaciones aritméticas, lógicas o relacionales. Los símbolos de agrupación están dados por (y).

```
var edad:int = (10 + 10) - 5; var  
bandera: bool = (5 == 5) == true; var  
bandera2: bool = (1 + 2) > (3 - 1);
```

5.7. Precedencia de Operaciones

La precedencia de operadores nos indica la importancia de que una operación debe realizarse por encima del resto. A continuación, se define la misma:

Nivel	Operador	Asociatividad
0	-	Derecha
1	**	No asociativa
2	*, /	Izquierda
3	+, -	Izquierda
4	==, !=, <, <=, >, >=	Izquierda
5	!	Derecha
6	^	Izquierda
7	&&	Izquierda
8		Izquierda

Nota: el nivel 0 es el nivel de mayor importancia

5.8. Caracteres de finalización y encapsulamiento de sentencias

El lenguaje se verá restringido por dos reglas que ayudan a finalizar una instrucción y encapsular sentencias:

- **Finalización de instrucciones:** para finalizar una instrucción se utilizará el signo ;

```
var edad:int = 18;
```

- **Encapsular sentencias:** para encapsular sentencias dadas por ciclos, métodos, funciones, etc, se utilizará los signos { y }.

```
if (true) {  
    var edad:int = 18;  
}
```

5.9. Declaración de variables

Una variable deberá de ser declarada antes de poder ser utilizada. Todas las variables tendrán un tipo de dato y un nombre de identificador. Las variables podrán ser declaradas global y localmente.

```
VAR <ID> : <TIPO> ;  
VAR <ID> : <TIPO> = <EXPRESION> ;  
  
var num : int;  
  
var nota : double;  
  
var num2 : int = 10;  
var nota2 : double = 20.6;
```

Las variables no pueden cambiar de tipo de dato, se deben mantener con el tipo declarado inicialmente, por lo que se debe de validar que el tipo de la variable y el valor sean compatibles.

5.10. Asignación de variables

Esta instrucción permitirá modificar el valor de la variable que fue previamente declarada, siempre y cuando sea posible modificarla.

```
var num : int;  
  
var nota : double;  
  
num = 10;  
nota = 20.6;
```

5.11. Casteos

Los casteos son una forma de indicar al lenguaje que convierta un tipo de dato en otro, por lo que, si queremos cambiar un valor a otro tipo, es la forma adecuada de hacerlo. Para hacer esto, se colocará la palabra reservada del tipo de dato destino entre paréntesis seguido de una expresión.

(<TIPO>) <EXPRESION>

El lenguaje aceptará los siguientes casteos:

- int a double
- double a int
- Int a String
- int a char
- Double a String
- char a int
- char a double

```
var edad : int = (int) 18.6;  
var letra : char = (char) 70;  
var numero: double = (double) 16;
```

Nota:

- La variable edad tomaría el valor de 18.
- La variable letra tomaría el valor de 'F' ya que 70 es el ascii de F.
- La variable número tomaría el valor de 16.0

5.12. Incremento y Decremento

Los incrementos y decrementos nos ayudan a realizar la suma o resta continua de un valor de uno en uno, es decir si incrementamos una variable, se incrementará de uno en uno, mientras que, si realizamos un decremento, hará la operación contraria.

```
<EXPRESION> + + ;  
<EXPRESION> - - ;  
  
var edad : int = 18;  
edad++; // tiene el valor de 19  
edad--; // tiene el valor de 18
```

5.13. Sentencias de control

Estas sentencias modifican el flujo del programa introduciendo condicionales. Las sentencias de control para el programa son el IF y el Match.

OBSERVACIONES:

- También, entre las sentencias pueden tener ifs anidados.

5.13.1. Sentencia IF

El lenguaje JavaUSAC posee la sentencia IF similar a otros lenguajes de programación, la cual permite ejecutar bloques de código se ejecuten si la condición a evaluarse es verdadera. Esta sentencia se define por las instrucciones if, else if, else.

Consideraciones

- Las instrucciones else if y else son opcionales
- La instrucción else if se puede utilizar tantas veces como se desee

```
if ( <EXPRESION> ) {  
    <INSTRUCCIONES>  
}  
|  
if ( <EXPRESION> ) {  
    <INSTRUCCIONES>  
} else {  
    <INSTRUCCIONES>  
}  
|  
if ( <EXPRESION> ) {  
    <INSTRUCCIONES>  
} else <IF>
```

/*

Son 3 variantes en total

1. IF
2. IF-ELSE
3. IF- ELSE IF

*/

//Ejemplo de implementar un if

```
if(x<50)  
{  
    Print("Menor que 50");  
    //Mássentencias  
}
```

//Ejemplo para implementar un if else

```
if(x<50)  
{  
    print("Menor que 50");  
    //Mássentencias  
}  
else  
{  
    print("Mayor que 100");  
    //Mássentencias  
}
```

```
//Ejemplo de como se implementa un else if
if (x > 50)
{
    print("Mayor que 50");
    //Más sentencias
}
else if (x <= 50 && x > 0)
{
    print ("Menor que 50");
}
if (x > 25)
{
    print("Número mayor que 25");
    //Más sentencias
}
else
{
    print("Número menor que 25");
    //Más sentencias
}
```

5.13.2. Switch Case

Switch case es una estructura utilizada para agilizar la toma de decisiones múltiples, trabaja de la misma manera que lo harían sucesivos if.

5.13.2.1. Switch

Estructura principal del switch, donde se indica la expresión a evaluar.

```
'switch' '(' [<EXPRESION> ] ')' '{'  
[<CASES_LIST>] [<DEFAULT>]  
'}'  
| 'switch' '(' [<EXPRESION> ] ')' '{'  
[<CASES_LIST>]  
'}'  
| 'switch' '(' [<EXPRESION> ] ')' '{'  
[<DEFAULT>]  
'}'
```

5.13.2.2. Case

Estructura que contiene las diversas opciones a evaluar con la expresión establecida en el switch.

```
'case' [<EXPRESION>] ':'  
[<INSTRUCCIONES>]
```

5.13.2.3. Default

Estructura que contiene las sentencias si en dado caso no haya salido del switch por medio de una sentencia break.

```
default :  
[<INSTRUCCIONES>]  
  
//EJEMPLODESWITCH  
intedad=18;  
switch(edad){  
Case10:  
    Print("Tengo 10 anios.");  
    //mas sentencias  
    Break;  
Case18:
```

```

        Print("Tengo 18 anios.");
        //mas sentencias
    Case25:
        Print("Tengo 25 anios.");
        //mas sentencias
        Break;
    Default:
        Print("No se que edad tengo. :(");
        //mas sentencias
        Break;
}
/*Salida esperada
Tengo 18anios.
No se que edad tengo. :(
*/

```

Nota: Si la cláusula “case” no posee ninguna sentencia “break”, al terminar todas las sentencias del case ingresado, el lenguaje seguirá evaluando las demás opciones.

5.14. Sentencias Cíclicas

Los ciclos o bucles son una secuencia de instrucciones de código que se ejecutan una vez tras otra mientras la condición, que se ha asignado para que pueda ejecutarse, sea verdadera. En el lenguaje actual, se podrán realizar 3 sentencias cíclicas que se describen a continuación.

Observaciones:

- Es importante destacar que pueden tener ciclos anidados entre las sentencias a ejecutar.
- También, entre las sentencias pueden tener ciclos diferentes anidados

5.14.1. While

El ciclo o bucle While, es una sentencia que ejecuta una secuencia de instrucciones mientras la condición de ejecución se mantenga verdadera.

```

while ( <EXPRESION> ) { <INSTRUCCIONES>
}

```

5.14.2. For

El ciclo o bucle for, es una sentencia que nos permite ejecutar N cantidad de veces la secuencia de instrucciones que se encuentra dentro de ella.

Observaciones:

- Para la actualización de la variable del ciclo for se puede utilizar
 - **Incremento | Decremento:** `i++ | i--`
 - **Asignación:** como `i = i+1`, `i = i-1`, etc, es decir, cualquier tipo de asignación
- Dentro pueden venir N instrucciones

```
for ( <ASIGNACIÓN> | <DECLARACION>; <CONDICIÓN> ;  
    <ACTUALIZACIÓN> ) { <INSTRUCCIONES>  
}
```

5.14.3. Do-While

El ciclo o bucle Do-While, es una sentencia que ejecuta al menos una vez el conjunto de instrucciones que se encuentran dentro de ella y que se sigue ejecutando mientras la condición sea verdadera.

Observaciones:

- Dentro pueden venir N instrucciones

```
do {  
    [ <INSTRUCCIONES> ]  
} while ( <EXPRESION> );
```

5.15. Sentencias de Transferencia

Las sentencias de transferencia nos permiten manipular el comportamiento de los bucles, ya sea para detenerlo o para saltarse algunas iteraciones. El lenguaje soporta las siguientes sentencias:

5.15.1. Break

La sentencia break hace que se salga del ciclo inmediatamente, es decir que el código que se encuentre después del break en la misma iteración no se ejecutara y este se saldrá del ciclo.

```
break ;

var i : int = 0;
for(i = 0; i < 3;i++){
    if (i==2){
        break; // me salgo en i = 2 y ya no se ejecuta lo demás
    }
    print(i);
}

// Salida
/*
0
1
*/
```

5.15.2. Continue

La sentencia continue puede detener la ejecución de la iteración actual y saltar a la siguiente. La sentencia continue siempre debe de estar dentro de un ciclo, de lo contrario será un error

```

continue ;

var i: int = 0;
for(i = 0; i <= 2;i++){
print(i);
    if (i==1){
        continue;
        /*me salte el resto de la iteración y continúe en i == 3 */
    } print(i*5);
}
// Salida
/*
0
0
1
2
10
*/

```

5.15.3. Return

La sentencia return finaliza la ejecución de un método o función y puede especificar un valor para ser devuelto a quien llama a la función.

```

return ;
return <EXPRESION> ;

// Ejemplos
return 10 *10;
return;
return 1;

```


5.16. Vectores

Los vectores son una estructura de datos de tamaño fijo que pueden almacenar valores de forma limitada. **El lenguaje permitirá únicamente el uso de arreglos de una o dos dimensiones.**

Observaciones:

- La posición de cada vector será N-1. Por ejemplo, si se quiere acceder al primer valor de un vector se accede como vector[0].

5.16.1. Declaración de Vectores

En esta declaración, se indica por medio de una lista de valores separados por coma, los valores que tendrá el vector, en este caso el tamaño del vector será el de la misma cantidad de valores de la lista.

```
<VAR> <ID> : <TIPO> [ ] = [ <LISTAVALORES> ] ;  
<VAR> <ID> : <TIPO> [ ] [ ] = [ <LISTAVALORES2> ] ;  
  
var vector1 : string [ ] = [ "Hola", "Mundo" ];  
var vector2 : int [ ] [ ] = [ [ 1, 2 ], [ 3, 4 ] ] ;
```

5.16.2. Acceso a vectores

Para acceder al valor de una expresión de un vector, se colocará el nombre del vector seguido del acceso.

```
<ID> [ <EXPRESION> ]  
  
var vector3:string[ ] = [ "Hola", "Mundo" ];  
  
var valor3 : string = vector3[0]; // Almacena el valor "hola"  
  
var vector4:int [ ] [ ] = [ [ 1, 2 ], [ 3, 4 ] ];  
var valor4:int = vector4[0][0]; // Almacena el valor 1
```

5.16.3. Asignación de Vectores

Para modificar el valor de una posición de un vector, se debe colocar el nombre del vector seguido del acceso y la expresión a asignar.

Observaciones:

- A una posición de un vector se le puede asignar el valor de otra posición de otro vector o del mismo vector.
- A una posición de un vector se le puede asignar el valor de una posición de una vector.

```
<ID> [ <EXPRESION> ] = <EXPRESION> ;  
<ID> [ <EXPRESION> ] [ <EXPRESION> ] = <EXPRESION> ;  
  
var vector3:string[] = ["Hola","Mundo"];  
vector3[0] = "OLC1";  
vector3[1] = "1er Semestre";
```

5.17. Listas Dinámicas

Las listas dinámicas son una estructura de datos de tamaño variable que pueden almacenar valores de forma ilimitada de un solo tipo: int, double, boolean, char o string.

5.17.1. Declaración de Listas

```
List< <TIPO> > <ID> = new List();  
  
List<int> miLista = new List();
```

5.17.2. Acceso a listas

Para acceder al valor de una expresión de una lista, se colocará el nombre de la lista seguido del acceso.

```
<ID> [ <EXPRESION> ]  
  
List<int> miLista = new List(); miLista.append(1); // el método  
append se explicará más adelante const valor:int = miLista[0]; //  
Almacena el valor 1
```

5.17.3. Asignación de Listas

Para modificar el valor de una posición de una lista, se debe colocar el nombre de la lista seguido de una expresión.

```
<ID> [ <EXPRESION> ] = <EXPRESION> ;
```

```
List<int> miLista = new List();  
miLista.append(1);  
miLista[0] = 2;
```

5.17.4. Append

Para agregar valores a la lista, se utiliza la palabra reservada append, la cual agrega el elemento al final de la lista.

```
<ID>.append( <EXPRESION> );
```

```
List<int> miLista = new List();  
  
miLista.append(1);
```

5.17.5. Remove

Para eliminar valores a la lista, se utiliza la palabra reservada remove, la cual recibe la posición del elemento a eliminar de la lista y lo retorna.

```
<ID>.remove( <EXPRESION> )
```

```
List<int> miLista = new List(); miLista.append(1);  
miLista.append(2); var miVar:int = miLista.remove(0);  
//almacena el valor de 1
```

5.18. Funciones

Una función es una subrutina de código que se identifica con un nombre, tipo y un conjunto de parámetros. Para este lenguaje las funciones serán declaradas definiendo primero su tipo, luego un identificador único, seguido de una lista de parámetros dentro de paréntesis (esta lista de parámetros puede estar vacía en el caso de que la función no utilice parámetros).

Cada parámetro debe estar compuesto por su tipo seguido de un identificador, para el caso de que sean varios parámetros se debe utilizar comas para separar cada parámetro y en el caso de que no se usen parámetros no se deberá incluir nada dentro de los paréntesis. Luego de definir la función y sus

parámetros se declara el cuerpo de la función, el cual es un conjunto de instrucciones delimitadas por llaves {}.

Para las funciones es obligatorio que las mismas posean un valor de retorno que coincida con el tipo con el que se declaró la función, en caso de que no sea el mismo tipo o de que no venga un retorno dentro del cuerpo de la función debería lanzarse un error de tipo semántico.

```
<TIPO> <ID> ( <PARAMETROS> ) {  
    <INSTRUCCIONES>  
}  
  
PARAMETROS → PARAMETROS , <TIPO> <ID>  
            | <TIPO> <ID>  
  
int conversion (int size,string tipo){  
    if(tipo=="metro"){  
        return size/3*3.281;  
    } else{  
        return -1;  
    }  
}
```

Cabe destacar que **no habrá sobrecarga de funciones y métodos** dentro de este lenguaje por lo que solo puede existir una función o método con el id declarado por lo que si se crea otra función o método con un id previamente utilizado esto debe de generar un error de tipo semántico.

5.19. Métodos

Un método también es una subrutina de código que se identifica con un tipo, nombre y un conjunto de parámetros, aunque a diferencia de las funciones estas subrutinas no deben de retornar un valor. Para este lenguaje los métodos serán declarados haciendo uso de la palabra reservada 'void', seguido de un identificador del método, seguido de una lista de parámetros dentro de paréntesis (esta lista de parámetros puede estar vacía en el caso de que la función no utilice parámetros).

Cada parámetro debe estar compuesto por su tipo seguido de un identificador, para el caso de que sean varios parámetros se debe utilizar comas para separar cada parámetro y en el caso de que no se usen parámetros no se

deberá incluir nada dentro de los paréntesis. Luego de definir el método y sus parámetros se declara el cuerpo del método, el cual es un conjunto de instrucciones delimitadas por llaves {}.

```
void <ID> ( <PARAMETROS> ) {  
    <INSTRUCCIONES>  
}
```

```
PARAMETROS → PARAMETROS , <TIPO><ID>  
            | <TIPO><ID>
```

Cabe destacar que **no habrá sobrecarga de funciones y métodos** dentro de este lenguaje por lo que solo puede existir una función o método con el id declarado por lo que si se crea otra función o método con un id previamente utilizado esto debe de generar un error de tipo semántico.

5.20. Llamadas

La llamada a una función específica la relación entre los parámetros reales y los formales y ejecuta la función. Los parámetros se asocian normalmente por posición, aunque, opcionalmente, también se pueden asociar por nombre.

La llamada a una función devuelve un resultado que ha de ser recogido, bien asignándole a una variable del tipo adecuado, bien integrándose en una expresión. La sintaxis de las llamadas de los métodos y funciones será la misma.

```
LLAMADA → <ID> ( <PARAMETROS_LLAMADA> )  
          | <ID> ( )
```

```
PARAMETROS_LLAMADA → PARAMETROS_LLAMADA , <EXPRESION>  
                    | <EXPRESION>
```

Observaciones:

- Al momento de ejecutar cualquier llamada, no se diferenciarán entre métodos y funciones, por lo tanto, podrá venir una función que retorne un valor como un método, pero la expresión retornada no se asignará a ninguna variable.
- Para la llamada a métodos como una instrucción se deberá de agregar el punto y coma (;) como una instrucción.

5.21. Función Print

Esta función nos permite imprimir expresiones y agrega un salto de línea al final del contenido

```
print ( <EXPRESION> ) ;  
  
print("Hola mundo");  
print(arreglo);  
  
print(arreglo[0]);  
  
print(e1.edad);
```

Función round

Esta función recibe como parámetro un valor numérico. Permite redondear los números decimales según las siguientes reglas:

- Si el decimal es mayor o igual a 0.5, se aproxima al entero superior. •
- Si el decimal es menor que 0.5, se aproxima al número inferior.

```
round ( <EXPRESION> )
```

5.22. Length

Esta función recibe como parámetro un vector, una lista o una cadena y devuelve el tamaño de este.

Observación:

- Si se usa para otro parámetro de tipo de dato no especificado, se considera un error semántico.

```
length ( <EXPRESION> )
```

5.23. ToString

Esta función permite convertir un valor de tipo numérico, caracter, bool o struct a texto.

Observación:

- Si se usa para otro parámetro de tipo de dato no especificado, se considera un error semántico.

```
toString ( <EXPRESION> )
```

5.27. Find

Esta función retorna true si la expresión existe dentro del vector o lista, caso contrario retorna false.

```
<ID>.Find ( <EXPRESION> )
```

```
// Ejemplo
```

```
var arr: int[] = [0, 1, 2]; const flag:bool = arr.Find(1); // almacena true
```

5.28. Función START

Para poder ejecutar todo el código generado dentro del lenguaje, se utilizará la sentencia START para poder indicar qué método o función es la que iniciará con la lógica del programa.

```
start <ID> ( ) ;  
start <ID> ( <PARAMETROS> ) ;
```

6. Reportes

Los reportes son parte fundamental de javaCraft, ya que muestra de forma visual las herramientas utilizadas para realizar la ejecución del código. A continuación, se muestran ejemplos de estos reportes. Queda a discreción del estudiante el diseño de estos, solo se pide que sean totalmente legibles.

6.27. Tabla de errores

El reporte de errores debe contener la información suficiente para detectar y corregir errores en el código fuente.

#	Tipo	Descripción	Línea	Columna
1	Léxico	El carácter “\$” no pertenece al lenguaje	5	3
2	Sintáctico	Se encontró Identificador y se esperaba Expresión.	6	3
3	Semántico	No se puede realizar resta entre CADENA y CADENA	8	10

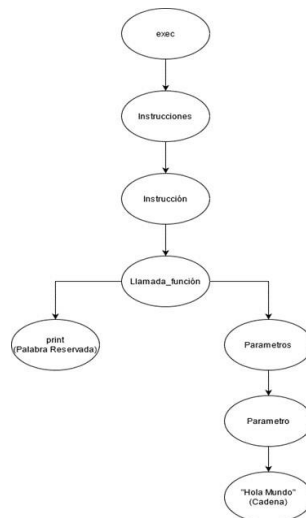
6.28. Tabla de Símbolos

Este reporte mostrará la tabla de símbolos después de la ejecución. Deberá mostrar las variables y arreglos declarados, así como su tipo, valor y toda la información que considere necesaria.

#	Id	Tipo	Tipo	Entorno	Valor	Línea	Columna
1	var	Variable	Entero	Funcion1	1	15	20
2	var2	Variable	Bool	Funcion2	true	20	13

6.29. AST

Este reporte muestra el AST producido al analizar los archivos de entrada. Este debe de representarse como un grafo. Se deben mostrar los nodos que el estudiante considere necesarios para describir el flujo realizado para analizar e interpretar sus archivos de entrada.



6.30. Salidas en consola

La consola es el área de salida del intérprete. Por medio de esta herramienta se podrán visualizar las salidas generadas por la instrucción “print”, así como los errores léxicos, sintácticos y semánticos.

```
> Este es un mensaje desde mi interprete de compi 1.
>
>
---> Error léxico: Símbolo "#" no reconocido en línea 10 y columna 7
---> Error Semántico: Se ha intentado asignar un entero a una variable booleana
```

7. Restricciones

- La aplicación deberá de desarrollarse utilizando el lenguaje JAVA.
- Las herramientas para realizar los analizadores serán JFLEX y CUP.
- **El proyecto debe ser realizado de forma individual.**
- **Copias completas/parciales** de: código, gramática, etc. serán merecedoras de una **nota de 0 puntos**, los responsables serán reportados al catedrático de la sección y a la Escuela de Ciencias y Sistemas.
- La calificación será sobre el archivo ejecutable de su aplicación, de cumplir con este requerimiento **NO SE CALIFICARÁ.**
- La entrega debe ser realizada mediante UEDI.

- El nombre del repositorio de Github debe ser **OLC1_VD25_#Carnet** -
Se debe agregar al auxiliar como colaborador del repositorio.
Usuario: LestherLopez
- Todas las salidas se deben de visualizar en la aplicación.

8. Entrega

El proyecto estará dividido en dos fases, a continuación, se define como se van a realizar las entregas

- FASE 1
 - Funcionalidades a entregar
 - Interfaz gráfica
 - Comentarios
 - Tipos de datos
 - Secuencias de escape
 - Expresiones (Aritméticas, lógicas, relacionales y casteos)
 - Signos de agrupación
 - Declaración de variables
 - Asignación de variables
 - Acceso de variables
 - Sentencias de control
 - Sentencias cíclicas
 - Sentencias de transferencia (menos el return)
 - Función Print
 - Reporte Tabla de Símbolos
 - Reporte de Errores ○

Entregables

- Código fuente de la aplicación
- Archivo ejecutable de la aplicación
- Archivo de gramática ○

Fecha de entrega

- **Martes 16 de diciembre hasta las 11:59 hrs**

- FASE 2
 - Funcionalidades a entregar
 - Vectores
 - Listas

- Funciones
- Métodos
- Llamadas
- Función Start
- Todos los reportes
- Resto de funciones nativas
- Entregables
 - Código fuente de la aplicación
 - Archivo de gramática

Fecha de entrega

- **Martes 30 de diciembre hasta las 11:59 hrs**